

---

Cloud Project

Image Rag Assistant

# *Image* RAG ASSISTANT

Relax & Let AI Organize Your Memories





# Core Problem

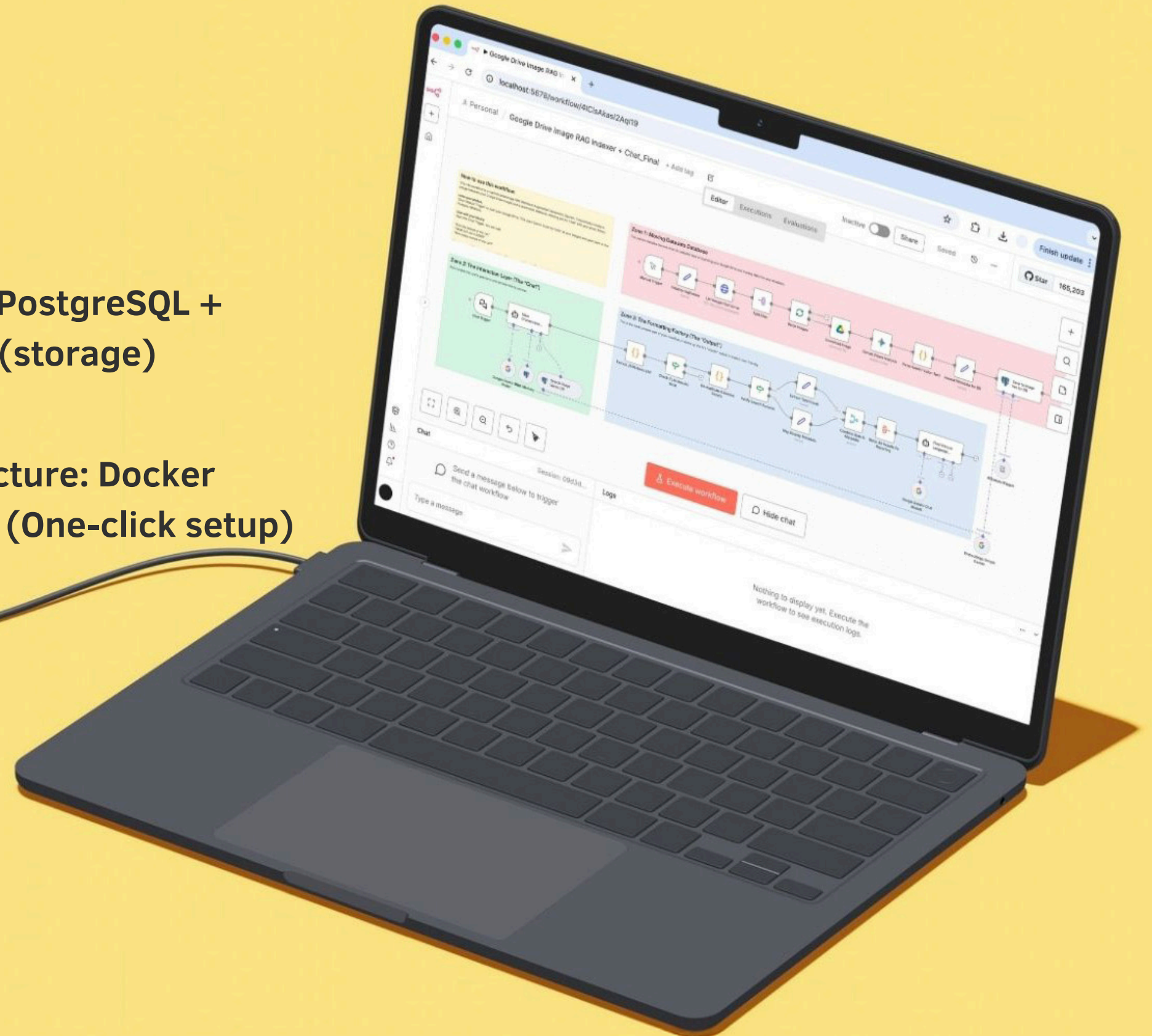
- Manual tagging is slow.
- Search is limited to dates and filenames.

Goal: Natural language retrieval and analysis.



# Tech Stack

- ✓ **Orchestration: n8n**  
(Low-code workflow engine)
- ✓ **Memory: PostgreSQL + pgvector** (storage)
- ✓ **Infrastructure: Docker Compose** (One-click setup)
- ✓ **Vision: Gemini 1.5 Flash**  
(VLM & Embeddings).
- ✓ **Memory: PostgreSQL + pgvector** (Semantic storage).



# System Overview

## How to use this workflow:

This n8n workflow is a sophisticated Image RAG (Retrieval-Augmented Generation) System. It essentially creates a bridge between your Google Drive images and a searchable database, allowing you to "chat" with your photo library.

### Index your photos:

Click 'Manual Trigger' to scan your Google Drive. This uses Gemini Vision to "look" at your images and save them to the Postgres database.

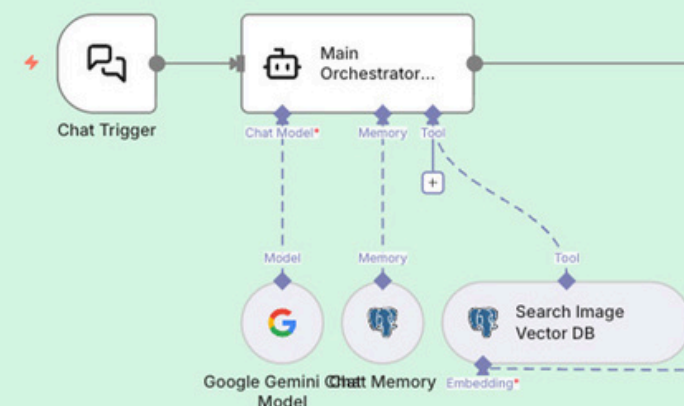
### Chat with your library:

Open the Chat Trigger. You can ask:

"Find the photos of my cat."  
"What did I do in 2020?"  
"How many photos of my cat?"

## Zone 2: The Interaction Layer (The "Chat")

This handles the user's questions and decides how to answer.



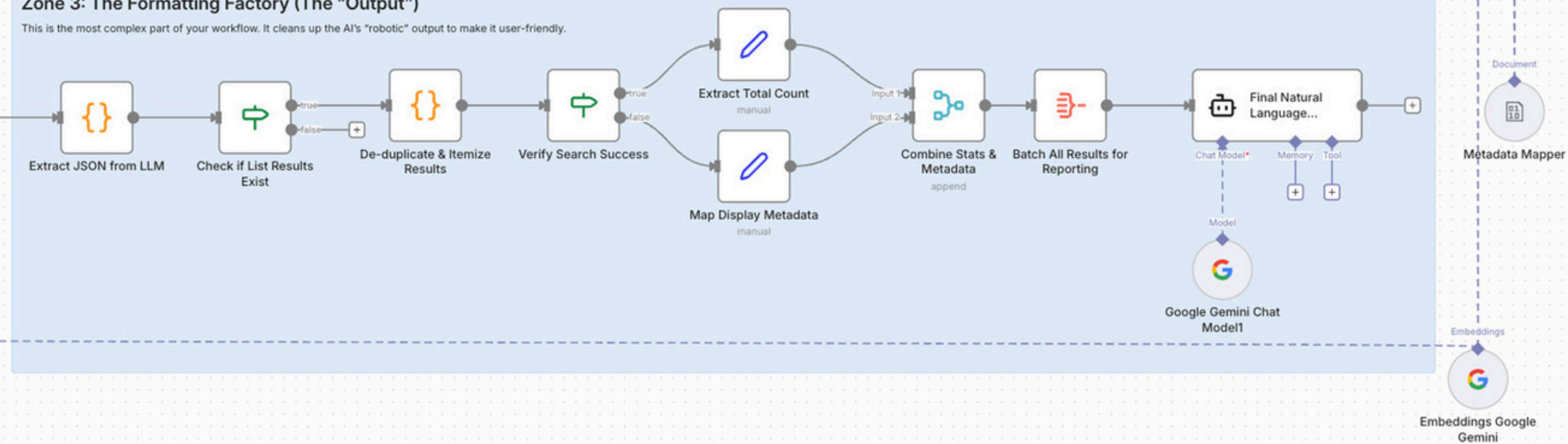
## Zone 1: Moving Data into Database

This section handles the one-time (or periodic) task of scanning your Google Drive and moving data into your database.



## Zone 3: The Formatting Factory (The "Output")

This is the most complex part of your workflow. It cleans up the AI's "robotic" output to make it user-friendly.





# Pipeline

## PIPELINE A: THE INDEXER

- Scan Google Drive.
- Gemini analyzes visual content.
- Convert descriptions into vectors.

### How to use this workflow:

This n8n workflow is a sophisticated Image RAG (Retrieval-Augmented Generation) System. It essentially creates a bridge between your Google Drive images and a searchable database, allowing you to "chat" with your photo library.

#### Index your photos:

Click "Manual Trigger" to scan your Google Drive. This uses Gemini Vision to "look" at your images and save them to the Postgres database.

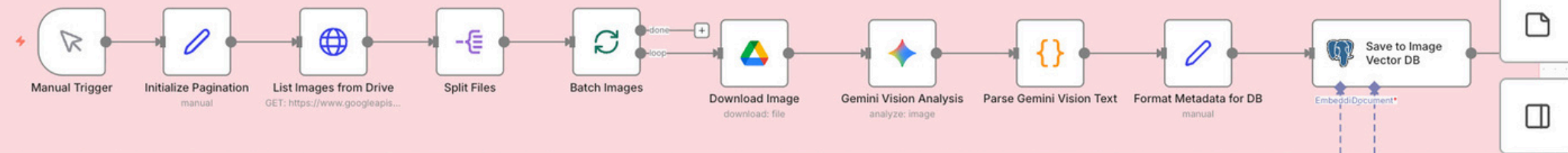
#### Chat with your library:

Open the Chat Trigger. You can ask:

"Find the photos of my cat."  
"What did I do in 2020?"  
"How many photos of my cat?"

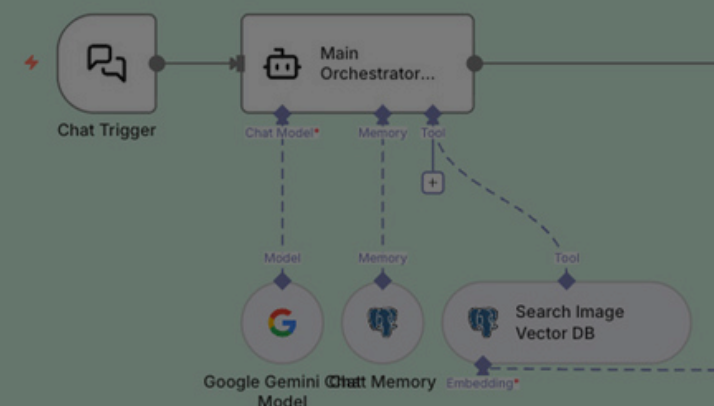
### Zone 1: Moving Data into Database

This section handles the one-time (or periodic) task of scanning your Google Drive and moving data into your database.



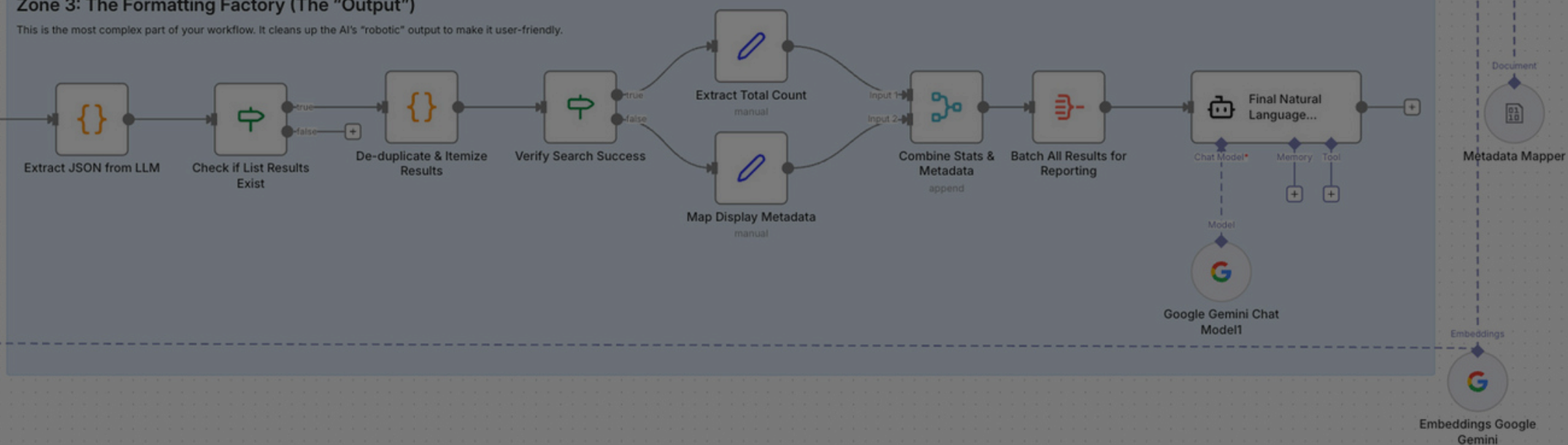
### Zone 2: The Interaction Layer (The "Chat")

This handles the user's questions and decides how to answer.



### Zone 3: The Formatting Factory (The "Output")

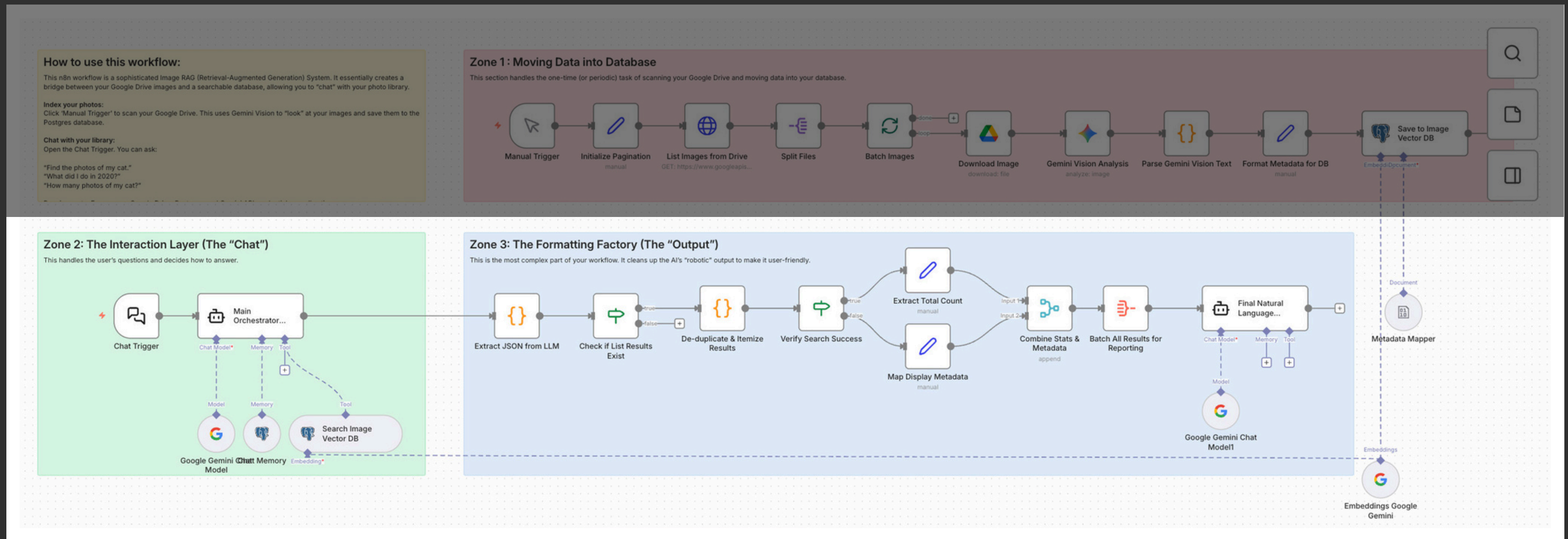
This is the most complex part of your workflow. It cleans up the AI's "robotic" output to make it user-friendly.



# Pipeline

## PIPELINE B: THE ASSISTANT

- User asks a question via Chat.
- Agent performs a vector search.
- Gemini synthesizes a "Story" with direct links.





---

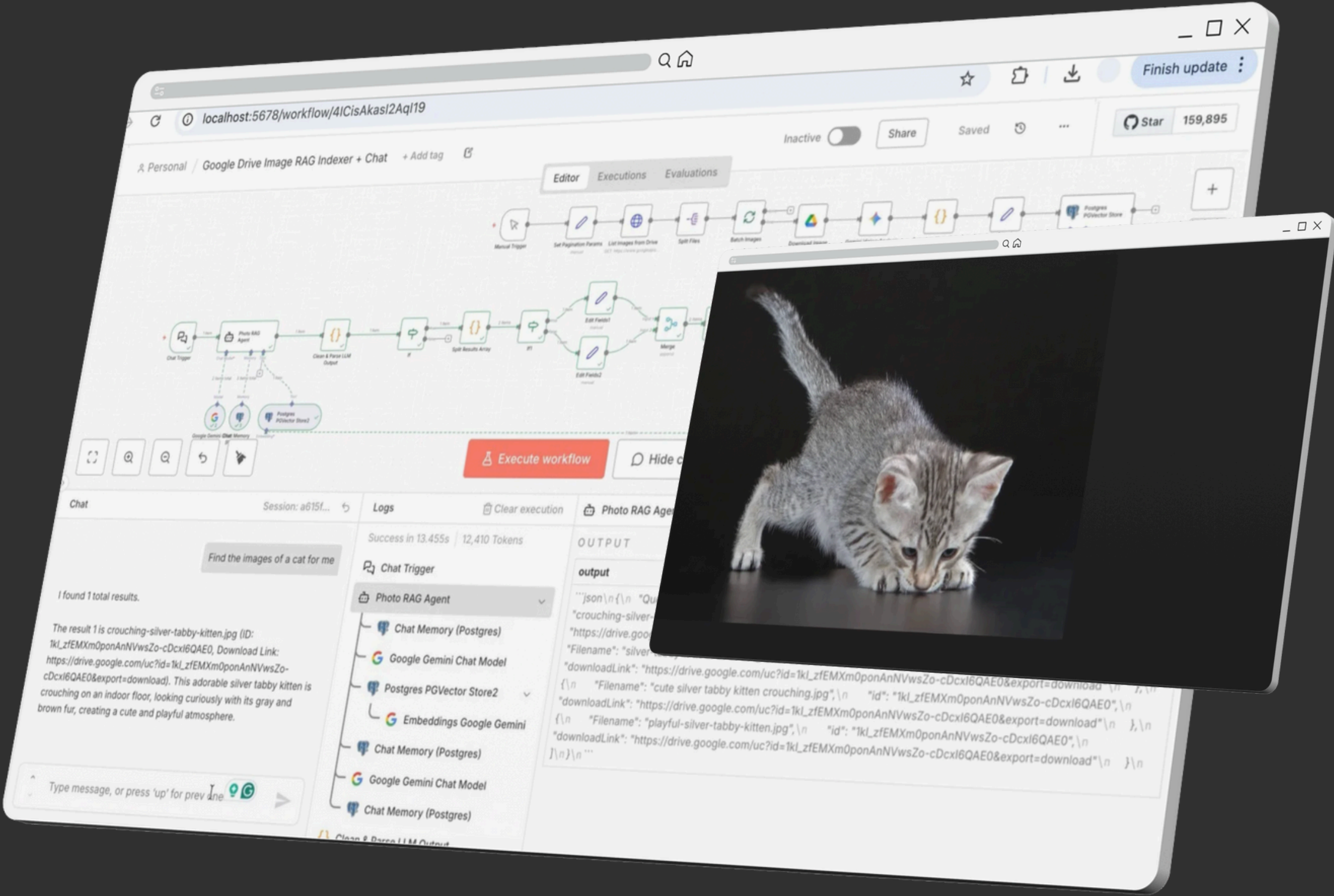
# Project Structure

image-rag-assistant/

- ├── **docker-compose.yml** # Configuration to launch Google Drive, n8n, Gemini, and DB services.
- ├── .env.example
- ├── README.md
- ├── architecturediagram.pdf
- ├── .gitignore
- ├── n8n-workflows/
  - └── **gdrive-image-rag-final.json** # The core RAG workflow.



# Demo Video



<https://youtu.be/DZQ9J00NTHQ>



09.

# Implementation Guide

THREE STEPS TO LAUNCH

THE IMPLEMENTATION GUIDE

## STEP 01: CONFIGURE

API keys in .env.

## STEP 02: DEPLOY

docker-compose up.

## STEP 03: IMPORT

Activate the n8n workflow.

# 10. Why This Matters

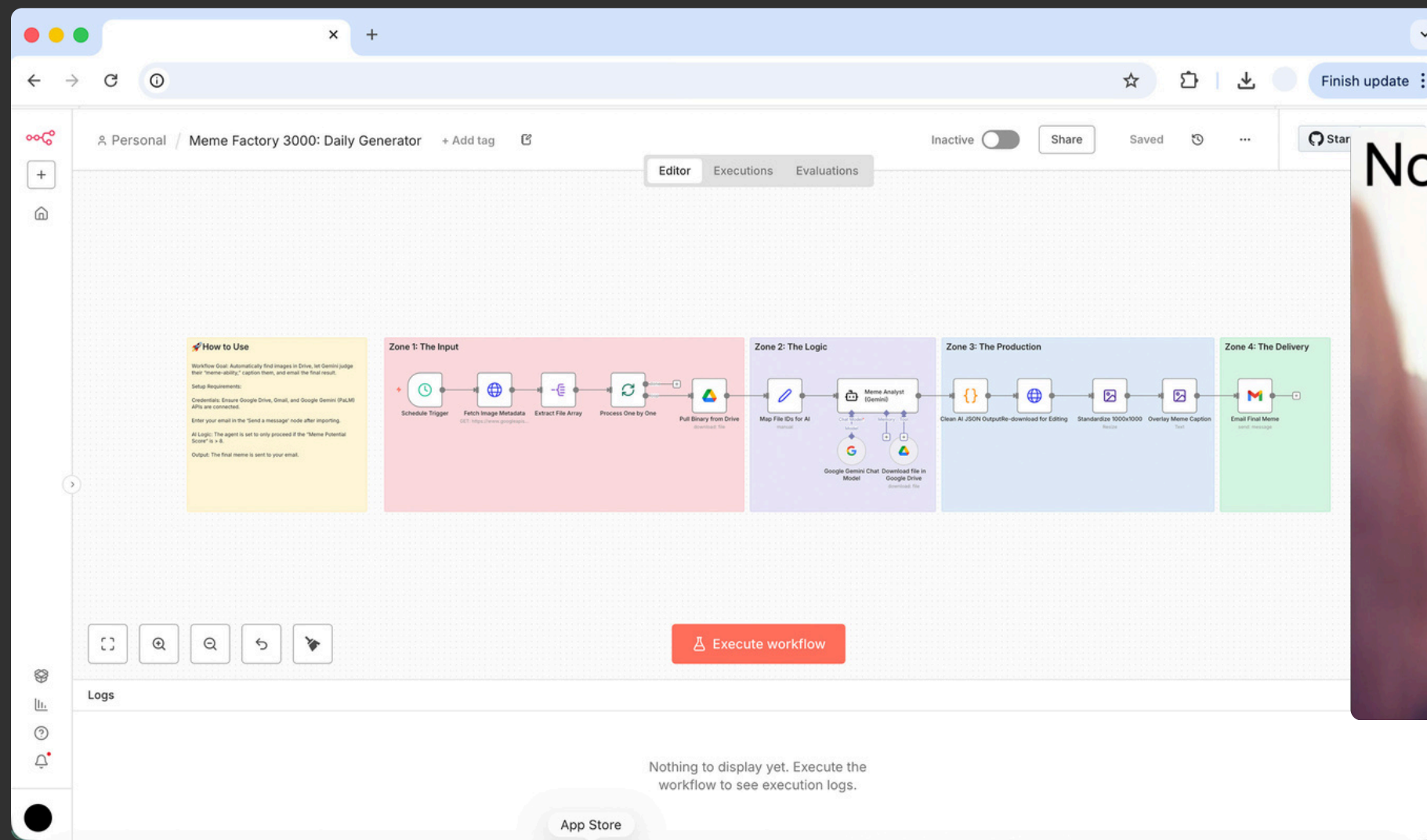
- **Automation:** Zero manual tagging required.
- **Intelligence:** Understands context
- **Scalability:** Works across any size Google Drive library.





# Meme Factory 3000

- Daily Scheduling: Set to trigger automatically every day at 9:00 AM.
- AI Curation: A Gemini AI Agent evaluates images for "Meme Potential" and only processes those with a score greater than 8.
- Gen-Z Humor: The system generates five captions, selects the best one, and overlays it onto a 1000x1000 resized image.
- Automatic Delivery: The final meme is emailed directly to the user via the Gmail node.



# Life Logger AI

- Visual Journaling: Uses Gemini 2.5 Flash to analyze Google Drive photos for emotions, activities, and locations.
- Total Context: The AI checks Google Calendar for schedules and OpenWeatherMap for the atmospheric "vibe" of the day.
- Memory: Photo information is stored in a PostgreSQL Vector database for intelligent retrieval.
- Hands-Free Writing: Automatically formats a first-person narrative and appends it to a Google Doc.

The screenshot displays the Life Logger AI interface, which is a web-based application for generating a daily journal. The interface is divided into several sections:

- Top Bar:** Includes a search bar, a toggle for "Inactive", a "Share" button, a "Saved" status, and a "Finish update" button. It also shows a star icon and a count of "165,623".
- Left Sidebar:** Contains a "Life Logger AI Setup Guide" and a "Chat" section with a session ID "fea71...".
- Main Content Area:** Features two workflow diagrams:
  - Zone 1: The Image Processing Pipeline:** A sequence of steps including "Schedule Trigger", "Set Pagination Params", "Fetch Daily Photos", "Split Files", "Process B at a Time", "Prepare Photo for AI", "Describe Photo Content", "Format AI Description", "Edit Fields", and "Save to Long-Term Memory".
  - Zone 2: The AI Agent Core:** A diagram showing the "Journal Writer Agent" interacting with various tools like "Agent Logic (Gemini)", "Google Docs Tool", "Conversation Memory1", and "Photo Memory Tool".
- Bottom Section:** Contains a "Chat" window with a session ID "fea71...", a "Logs" section showing "Success in 16.718s | 28,169 Tokens", and an "INPUT" section with fields for "sessionId", "action", and "chatInput".
- Right Panel:** Displays a generated journal entry titled "A Quiet Comfort on the Couch" for December 30, 2025. The content describes a peaceful evening spent with a cat, highlighting the user's appreciation for simple moments.





# Thank You

Now let's start building something great.

